CHAPTER 2

PIC ARCHITECTURE & ASSEMBLY LANGUAGE PROGRAMMING

SECTION 2.8:

Harvard architecture in the PIC

Every microprocessor must have memory space to store program (code) and data. As we have seen so far, the PIC is no exception with its code ROM space and data RAM (file register) space. While code provides instructions to the CPU, the data provides the information to be processed. The CPU uses buses (wire traces) to access the code ROM and data RAM memory spaces. The early computers used the same bus for accessing both the code and data. Such an architecture is commonly referred to as von Neumann (Princeton) architecture. That means for von Neumann computers, the process of accessing the code or data could cause them to get in each other's way and slow down the processing speed of the CPU, because each had to wait for the other to finish fetching. To speed up the process of program execution, some CPUs use what is called Harvard architecture. In Harvard architecture, we have separate buses for the code and data memory. That means that we need four sets of buses: (1) a set of data buses for

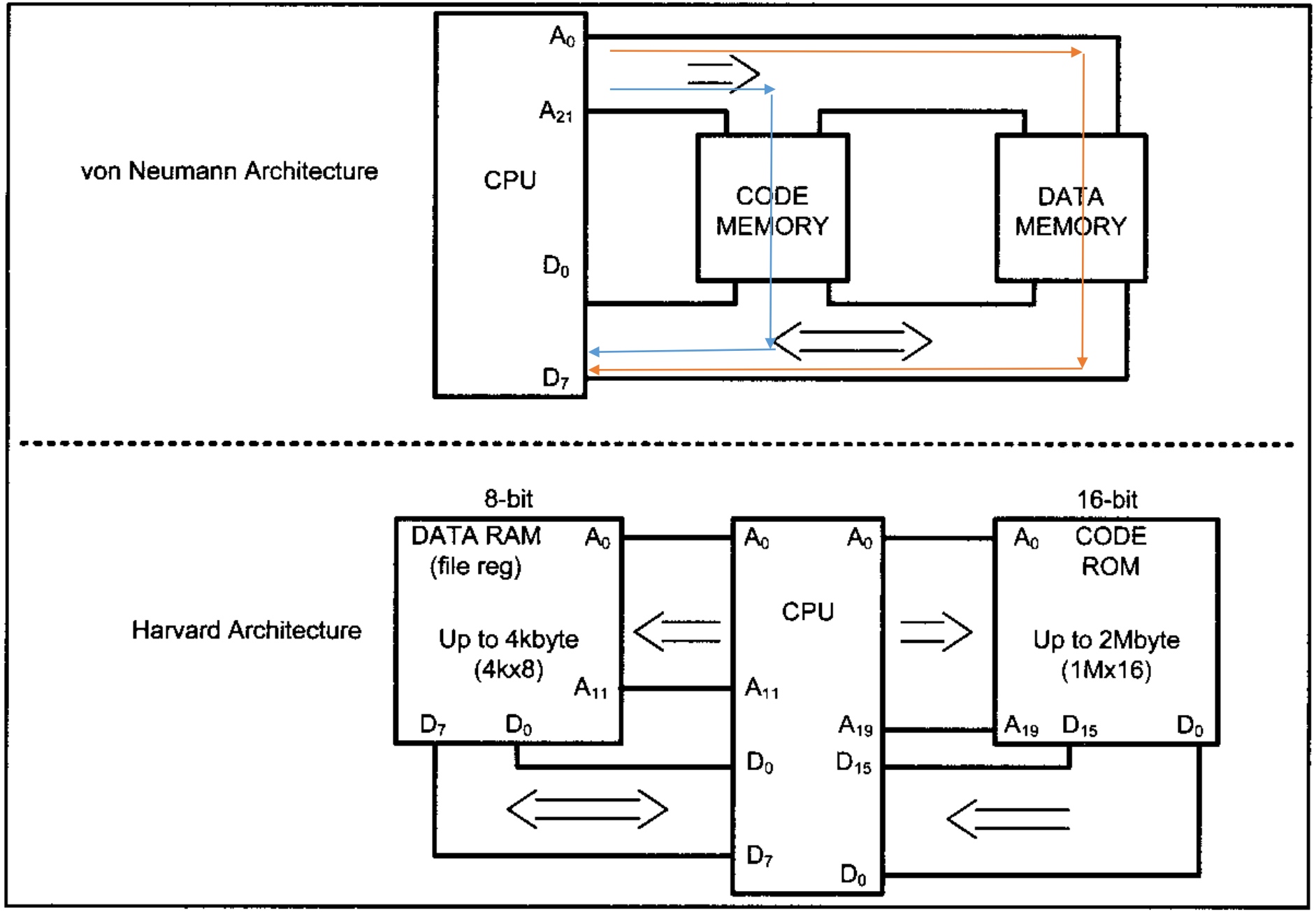


Figure 2-14. von Neumann vs. Harvard Architecture

carrying data into and out of the CPU, (2) a set of address buses for accessing the data, (3) a set of data buses for carrying code into the CPU, and (4) an address bus for accessing the code. See Figure 2-14. This is easy to implement inside an IC chip such as a microcontroller where both ROM code and data RAM are internal (on-chip) and distances are on the micron and millimeter scale. But to implement Harvard architecture for systems such as x86 IBM PC-type computers is very expensive because the RAM and ROM that hold code and data are external to the CPU. Separate wire traces for data and code on the motherboard will make the board large and expensive. For example, for a Pentium microprocessor with a 64-bit data bus and a 32-bit address bus we will need about 100 wire traces on the mother board if it is von Neumann architecture (96 for address and data, plus a few others for control signals of read and write and so on). But the number of wire traces will double to 200 if we use Harvard architecture. Harvard architecture will also necessitate a large number of pins coming out of the microprocessor itself. For this reason you do not see Harvard architecture implemented in the world of PCs and workstations. This is also the reason that microcontrollers such as PIC use Harvard architecture internally, but they still use von Neumann architecture if they need external memory for code and data space. The von Neumann architecture was developed at Princeton University, while the Harvard architecture was the work of Harvard University.

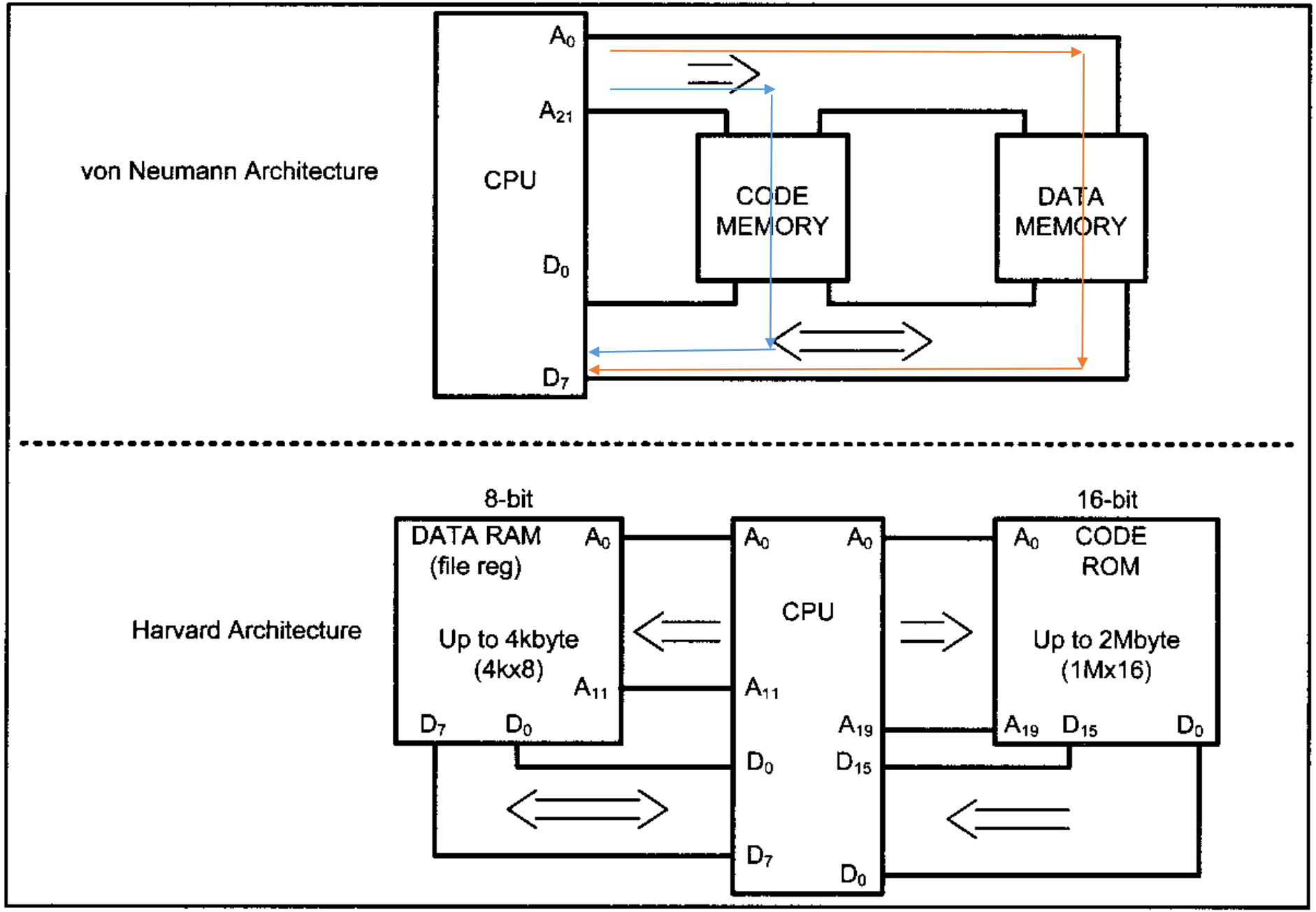


Figure 2-14. von Neumann vs. Harvard Architecture

RISC architecture

In the early 1980s, a controversy broke out in the computer design community, but unlike most controversies, it did not go away. Since the 1960s, in all mainframes and minicomputers, designers put as many instructions as they could think of into the CPU. Some of these instructions performed complex tasks. An example is adding data memory locations and storing the sum into memory. Naturally, microprocessor designers followed the lead of minicomputer and mainframe designers. Because these microprocessors used such a large number of instructions and many of them performed highly complex activities, they came to be known as CISC (complex instruction set computer). According to several studies in the 1970s, many of these complex instructions etched into the brain of the CPU were never used by programmers and compilers. The huge cost of implementing a large number of instructions (some of them complex) into the microprocessor, plus the fact that a good portion of the transistors on the chip are used by the instruction decoder, made some designers think of simplifying and reducing the number of instructions. As this concept developed, the resulting processors came to be known as RISC (reduced instruction set computer).

SECTION 2.9: RISC ARCHITECTURE IN THE PIC

There are three ways available to microprocessor designers to increase the processing power of the CPU:

1. Increase the clock frequency of the chip. One drawback of this method is that the higher the frequency, the more power and heat dissipation. Power and heat dissipation is especially a problem for hand-held devices.
2. Use Harvard architecture by increasing the number of buses to bring more information (code and data) into the CPU to be processed. While in the case of x86 and other general purpose microprocessors this architecture is very expensive and unrealistic, in today's microcontrollers this is not a problem. As we saw in the last section, the PIC18 has Harvard architecture.
3. Change the internal architecture of the CPU and use what is called RISC architecture.

1. Increase the Clock Frequency of the Chip

The clock frequency determines how many instructions the CPU can execute per second. A higher clock frequency means the CPU can perform calculations and move data faster, leading to a direct increase in processing power.

Example:

Imagine a CPU with a clock speed of 1 GHz (1 billion cycles per second).

- At 1 GHz, the CPU completes 1 billion instructions per second.
- If a designer increases the clock speed to 3 GHz, the CPU can now complete 3 billion instructions per second, theoretically tripling its processing power.
- The main drawback is that a faster clock requires more electrical power and generates more heat. This is a critical concern for devices with limited cooling, such as laptops, tablets, and smartphones, as it can lead to overheating and shorter battery life.

CHAPTER 3

BRANCH, CALL, AND TIME DELAY LOOP

SECTION 3.1: BRANCH INSTRUCTIONS AND LOOPING

In this section we first discuss how to perform a looping action in PIC and then the branch (jump) instructions, both conditional and unconditional.

Looping in PIC

Repeating a sequence of instructions or an operation a certain number of times is called a *loop*. The loop is one of most widely used programming techniques. In the PIC, there are several ways to repeat an operation many times. One way is to repeat the operation over and over until it is finished, as shown below:

```
MOVLW 0       ;WREG = 0
ADDLW 3       ;add value 3 to WREG
ADDLW 3       ;add value 3 to WREG(W = 6)
ADDLW 3       ;add value 3 to WREG(W = 9)
ADDLW 3       ;add value 3 to WREG(W = 0Ch)
ADDLW 3       ;add value 3 to WREG(W = 0Fh
```

In the above program, we add 3 to WREG 5 times. That makes $5 \times 3 = 15 = 0Fh$. One problem with the above program is that too much code space would be needed to increase the number of repetitions to 50 or 100. A much better way is to use a loop. There are two ways to do a loop in PIC. Next, we describe each method.

DECFSZ instruction and looping

The DECFSZ (decrement fileReg skip zero) instruction is a widely used instruction supported across all PIC families of microcontrollers from PIC12 to PIC18. It has the following format:

```
DECFSZ   fileReg , d   ;decrement fileReg and skip next instruction if 0
```

In this instruction, the fileReg is decremented, and if its content is zero, it skips the next instruction. By placing the "GOTO target" instruction right below it we can create a loop. The target address of the "GOTO target" instruction is the beginning of the loop, as shown in Examples 3-1 and 3-2. Figure 3-1 shows the flowchart for the DECFSZ instruction. Study the flowchart structure in Appendix D to get familiar with the symbols. The flowchart is a widely used method to represent a sequence of actions pictorially. Its usage for program design is recommended very strongly.

Example 3-1

Write a program to (a) clear WREG, and (b) add 3 to WREG ten times and place the result in SFR of PORTB. Use the DECFSZ instruction to perform looping.

Example 3-1

Write a program to (a) clear WREG, and (b) add 3 to WREG ten times and place the result in SFR of PORTB. Use the DECFSZ instruction to perform looping.

Solution:

```
;this program adds value 3 to WREG ten times

COUNT   EQU 0x25              ;use loc 25H for counter

        MOVLW   d'10'         ;WREG = 10 (decimal) for counter
        MOVWF   COUNT         ;load the counter
        MOVLW   0             ;WREG = 0
AGAIN   ADDLW   3             ;add 03 to WREG (WREG = sum)
        DECFSZ  COUNT,F       ;decrement counter, skip if count = 0
        GOTO    AGAIN         ;repeat until count becomes 0
        MOVWF   PORTB         ;send sum to PORTB SFR
```

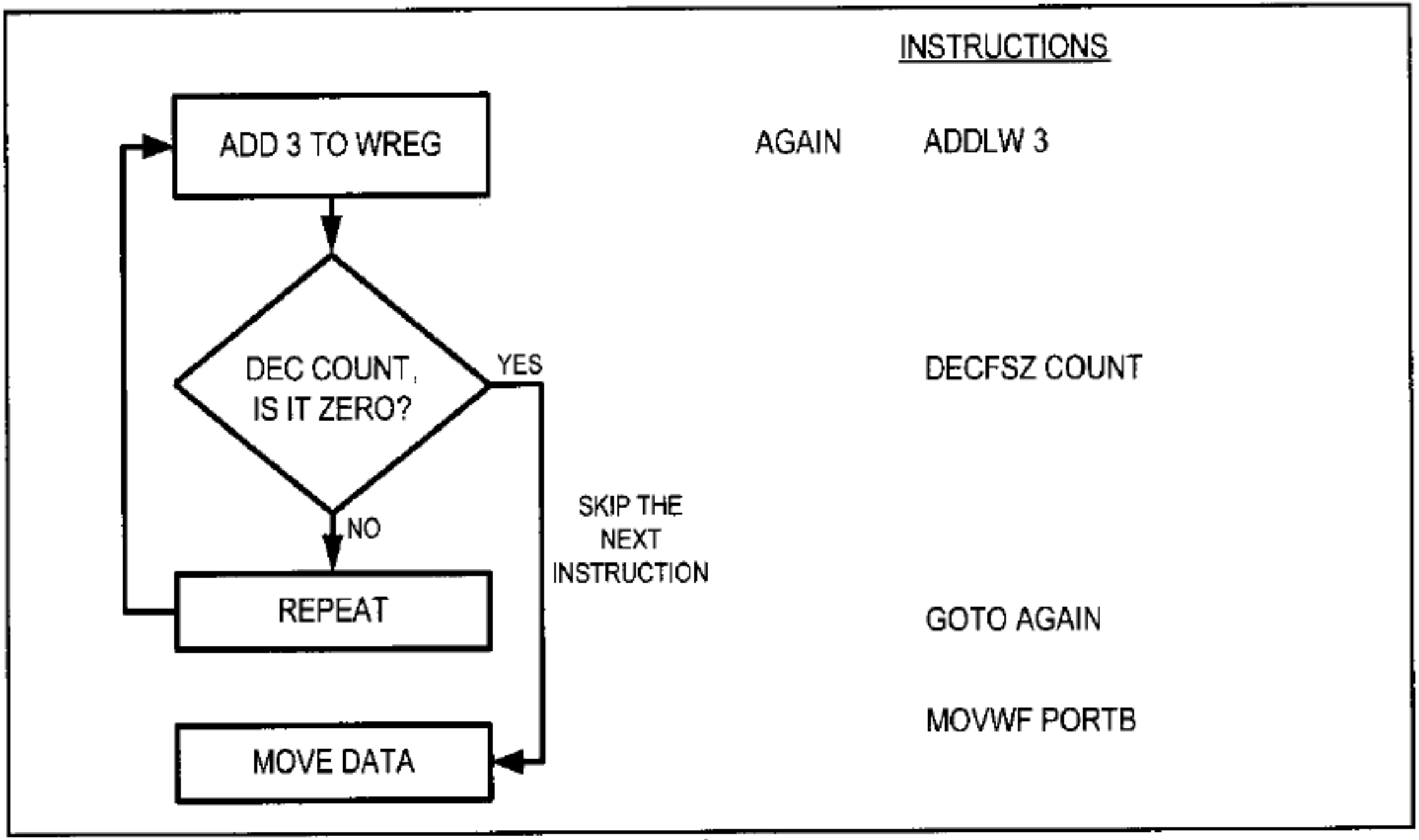


Figure 3-1. Flowchart for the DECFSZ Instruction

Example 3-2

Write a program to (a) clear WREG, then (b) add 3 to WREG ten times.

Use the zero flag and BNZ.

Solution:

Example 3-2

Write a program to (a) clear WREG, then (b) add 3 to WREG ten times.

Use the zero flag and BNZ.

Solution:

```
;this program adds value 3 to the WREG ten times

      COUNT EQU 0x25     ;use loc 25H for counter

      MOVLW d'10'        ;WREG = 10 (decimal) for counter
      MOVWF COUNT        ;load the counter
      MOVLW 0            ;WREG = 0
AGAIN ADDLW 3            ;add 03 to WREG (WREG = sum)
      DECF COUNT, F      ;decrement counter
      BNZ AGAIN          ;repeat until COUNT = 0
      MOVWF PORTB        ;send sum to PORTB SFR
```

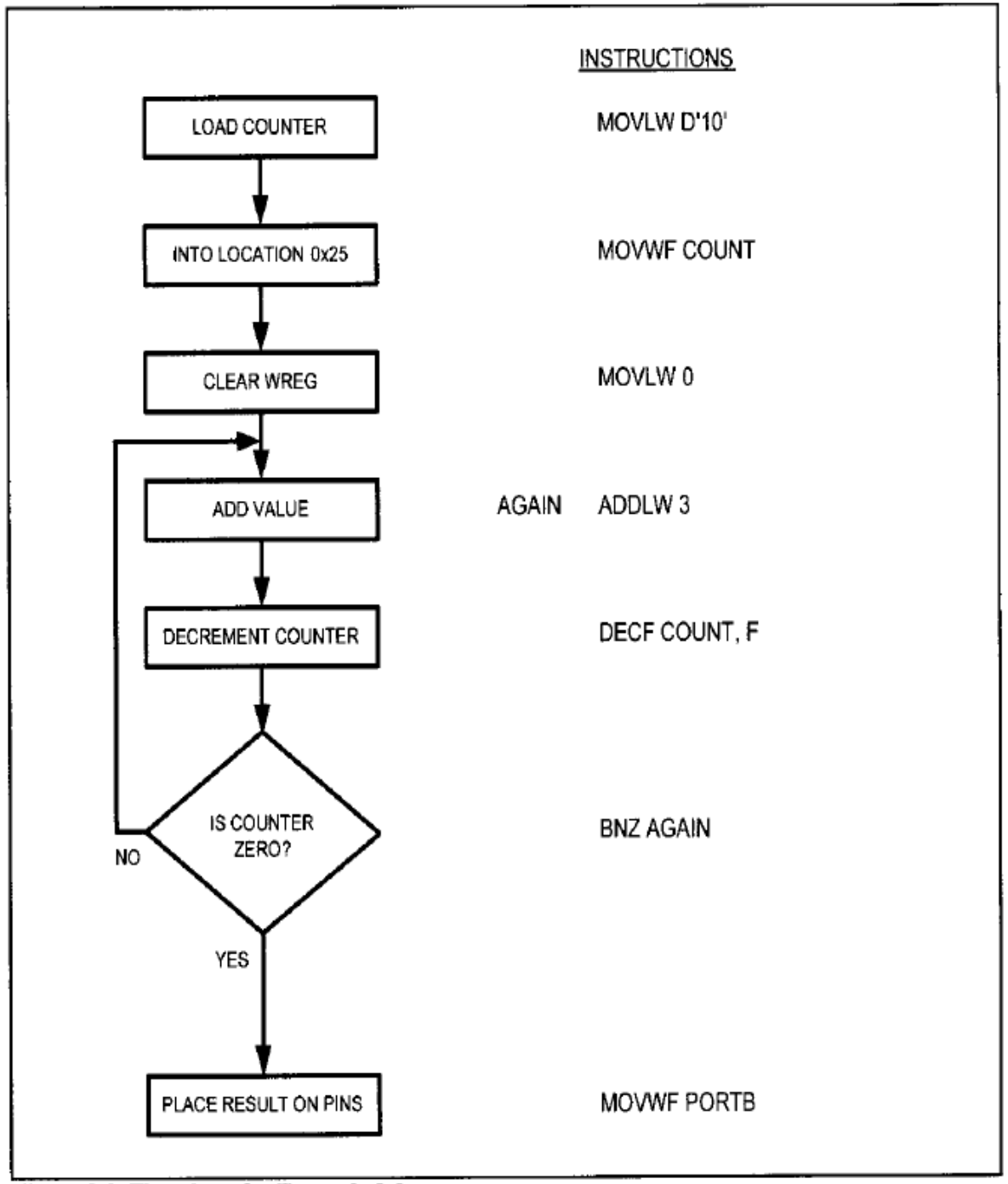


Figure 3-2. Flowchart for Example 3-2

Example 3-3

What is the maximum number of times that the loop in Example 3-2 can be repeated?

Solution:

Example 3-3

What is the maximum number of times that the loop in Example 3-2 can be repeated?

Solution:

Because location COUNT in fileReg is an 8-bit register, it can hold a maximum of FFH (255 decimal); therefore, the loop can be repeated a maximum of 255 times. See Example 3-4 to bypass this limitation.

Loop inside a loop

As shown in Example 3-2, the maximum count is 255. What happens if we want to repeat an action more times than 255? To do that, we use a loop inside a loop, which is called a *nested loop*. In a nested loop, we use two registers to hold the count. See Example 3-4.

Example 3-4

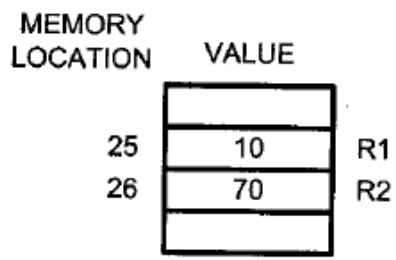
Write a program to (a) load the PORTB SFR register with the value 55H, and (b) complement Port B 700 times.

Solution:

Because 700 is larger than 255 (the maximum capacity of any register), we use two registers to hold the count. The following code shows how to use fileReg locations 25H and 26H as a register for counters.

```
      R1 EQU 0x25
      R2 EQU 0x26
      COUNT_1 EQU d'10'
      COUNT_2 EQU d'70'
      MOVLW 0x55          ;WREG = 55h
      MOVWF PORTB         ;PORTB = 55h
      MOVLW COUNT_1       ;WREG = 10, outer loop count value
      MOVWF R1            ;load 10 into loc 25H (outer loop count)
LOP_1 MOVLW COUNT_2       ;WREG = 70, inner loop count value
      MOVWF R2            ;load 70 into loc 26H
LOP_2 COMPF PORTB, F      ;complement Port B SFR
      DECF R2, F          ;dec fileReg loc 26 (inner loop)
      BNZ LOP_2           ;repeat it 70 times
      DECF R1, F          ;dec fileReg loc 25 (outer loop)
      BNZ LOP_1           ;repeat it 10 times
```

In this program, fileReg location 0x26 is used to keep the inner loop count. In the instruction "`BNZ LOP_2`", whenever location 26H becomes 0 it falls through and "`DECF R1, F`" is executed. This instruction forces the CPU to load the inner count with 70 if it is not zero, and the inner loop starts again. This process will continue until location 25 becomes zero and the outer loop is finished.



DIY

Looping 100,000 times

Because two registers give us a maximum value of 65025 (255 × 255 = 65025), we can use three registers to get up to more than 16 million (2^{24}) iterations. The following code repeats an action 100,000 times:

```
R1  EQU  0x1                ;assign RAM loc for the R1-R2
R2  EQU  0x2
R2  EQU  0x3
COUNT_1  EQU  D'100'        ;fixed value for 100,000 times
COUNT_2  EQU  D'100'
COUNT_3  EQU  D'10'

        MOVLW  0x55
        MOVWF  PORTB
        MOVLW  COUNT_3
        MOVWF  R3
LOP_3   MOVLW  COUNT_2
        MOVWF  R2
LOP_2   MOVLW  COUNT_1
        MOVWF  R1
LOP_1   COMPF  PORTB, F
        DECF   R1 ,F
        BNZ    LOP_1
        DECF   R2, F
        BNZ    LOP_2
        DECF   R3, F
        BNZ    LOP_3
```

Other conditional jumps

Conditional branches for the PIC are summarized in Table 3-1. More details of each instruction are provided in Appendix A. In Table 3-1, notice that some of the instructions, such as BZ (Branch if Z = 1) and BC (Branch if C = 1), jump only if a certain condition is met. Next, we examine some conditional branch instructions with examples.

Table 3-1: PIC Conditional Branch (Jump) Instructions

Instruction	Action
BC	Branch if C = 1
BNC	Branch if C ≠ 0
BZ	Branch if Z = 1
BNZ	Branch if Z ≠ 0
BN	Branch if N = 1
BNN	Branch if N ≠ 0
BOV	Branch if OV = 1
BNOV	Branch if OV ≠ 0

BZ (Branch if Z = 1)

In this instruction, the Z flag is checked. If it is high, it jumps to the target address. For example, look at the following code.

```
OVER   MOVF  PORTB,W      ;read Port B and put it in WREG
       JZ OVER            ;jump if WREG is zero
```

In this program, if PORTB is zero, it jumps to the label OVER. It stays there until PORTB has a value other than zero. Notice that the BZ instruction can be used to see whether any fileReg or WREG is zero. More importantly, you don't have to perform an arithmetic instruction such as decrement to use the BZ instruction. See Example 3-5.

Example 3-5

Write a program to determine if fileReg location 0x30 contains the value 0. If so, put 55H in it.

Example 3-5

Write a program to determine if fileReg location 0x30 contains the value 0. If so, put 55H in it.

Solution:

```
        MYLOC EQU 0x30
        MOVF  MYLOC,F              ;copy MYLOC to itself
        BNZ   NEXT                 ;branch if MYLOC is not zero
        MOVLW 0x55
        MOVWF MYLOC                ;put 0x55 if MYLOC has zero value
NEXT    ...
```

BNC (branch if no carry, branch if CY = 0)

In this instruction, the carry flag bit in the Status register is used to make the decision whether to jump. In executing "`BNC label`", the processor looks at the carry flag to see if it is raised (C = 1). If it is not, the CPU starts to fetch and execute instructions from the address of the label. If C = 1, it will not branch but will execute the next instruction below BNC. Study Example 3-6 to see how BNC is used to add numbers together when the sum is higher than FFH. Note that there

Example 3-6

Find the sum of the values 79H, F5H, and E2H. Put the sum in fileReg locations 5 (low byte) and 6 (high byte).

Solution: First, perform addition manually on paper.

- The first addition is $79H + F5H$.
- In decimal, that's $121 + 245 = 366$.
- In binary, this addition looks like:

```
    0111 1001   (79H)
  + 1111 0101   (F5H)
  -----------
1 0110 1110   (16EH)
```

The sum, $16EH$, requires nine bits. The lower 8 bits ($6EH$) are stored in the main register, and the **9th bit, the `1` on the far left, is the carry-out.**

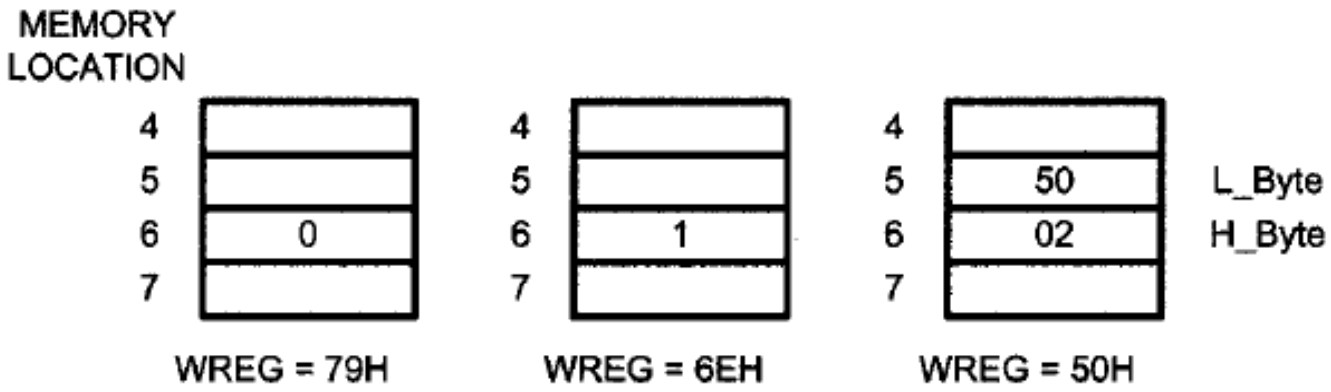
Example 3-6

Find the sum of the values 79H, F5H, and E2H. Put the sum in fileReg locations 5 (low byte) and 6 (high byte).

Solution:

```
L_Byte EQU 0x5          ;assign RAM loc 5 to L_byte of sum
H_Byte EQU 0x6          ;assign RAM loc 6 to H_byte of sum

       ORG 0h
       MOVLW 0x0        ;clear WREG(WREG = 0)
       MOVWF H_Byte     ;H_Byte = 0
       ADDLW 0x79       ;WREG = 0 + 79H = 79H, C = 0
       BNC   N_1        ;if C = 0, add next number
       INCF  H_Byte,F   ;C = 1, increment (now H_Byte = 0)
N_1    ADDLW 0xF5       ;WREG = 79 + F5 = 6E and C = 1
       BNC   N_2        ;branch if CY = 0
       INCF  H_Byte,F   ;C = 1, increment (now H_Byte = 1)
N_2    ADDLW 0xE2       ;WREG = 6E + E2 = 50 and C = 1
       BNC   OVER       ;branch if C = 0
       INCF  H_Byte,F   ;C = 1, increment (now H_Byte = 2)
OVER   MOVWF L_Byte     ;now L_Byte = 50H, and H_Byte = 02
       END
```



- The first addition is $79H + F5H$.
- In decimal, that's $121 + 245 = 366$.
- In binary, this addition looks like:

```
  0111 1001   (79H)
+ 1111 0101   (F5H)
----------
1 0110 1110   (16EH)
```

The sum, $16EH$, requires nine bits. The lower 8 bits ($6EH$) are stored in the main register, and the **9th bit, the `1` on the far left, is the carry-out.**

Example 3-9

Toggle all the bits of the SFR register of Port B by sending to it the values 55H and AAH continuously. Put a time delay in between each issuing of data to Port B.

Example 3-9

Toggle all the bits of the SFR register of Port B by sending to it the values 55H and AAH continuously. Put a time delay in between each issuing of data to Port B.

Solution:

```
MYREG EQU   0x08                ;use location 08 as counter
          ORG       0
BACK      MOVLW     0x55        ;load WREG with 55H
          MOVWF     PORTB       ;send 55H to port B
          CALL      DELAY       ;time delay
          MOVLW     0xAA        ;load WREG with AA (in hex)
          MOVWF     PORTB       ;send AAH to port B
          CALL      DELAY
          GOTO      BACK        ;keep doing this indefinitely
;———————   this is the delay subroutine
          ORG       300H        ;put time delay at address 300H
DELAY     MOVLW     0xFF        ;WREG = 255,the counter
          MOVWF     MYREG
AGAIN     NOP                   ;no operation wastes clock cycles
          NOP
          DECF      MYREG, F
          BNZ       AGAIN       ;repeat until MYREG becomes 0
          RETURN                ;return to caller
          END                   ;end of asm file
```